Proof Assistant e Fondamenti: la Matematica nell'Era Digitale

Nirvana Coppola

Università di Padova

14/03/2026

A che serve formalizzare la matematica?

Cos'è una dimostrazione matematica?

A che serve formalizzare la matematica?

Cos'è una dimostrazione matematica?

- Una successione di deduzioni logiche svolte a partire da assiomi e fatti noti.

A che serve formalizzare la matematica?

Cos'è una dimostrazione matematica?

- Una successione di deduzioni logiche svolte a partire da assiomi e fatti noti.
- Una serie di frasi che hanno convinto il peer reviewer.

A che serve formalizzare la matematica?

Cos'è una dimostrazione matematica?

- Una successione di deduzioni logiche svolte a partire da assiomi e fatti noti.
- Una serie di frasi che hanno convinto il peer reviewer.

A che serve formalizzare la matematica?

Cos'è una dimostrazione matematica?

- Una successione di deduzioni logiche svolte a partire da assiomi e fatti noti.
- Una serie di frasi che hanno convinto il peer reviewer.

Ma errare è umano!

Come facciamo a formalizzare della matematica?

Un *proof assistant* o *theorem prover interattivo* è un programma per scrivere dimostrazioni formali.

Come facciamo a formalizzare della matematica?

Un *proof assistant* o *theorem prover interattivo* è un programma per scrivere dimostrazioni formali.

Come?

Come facciamo a formalizzare della matematica?

Un *proof assistant* o *theorem prover interattivo* è un programma per scrivere dimostrazioni formali.

Come?

- Agli occhi dell'utente: l'utente scrive la dimostrazione in un linguaggio che il theorem prover capisce. Se compila, la dimostrazione è giusta!

Come facciamo a formalizzare della matematica?

Un *proof assistant* o *theorem prover interattivo* è un programma per scrivere dimostrazioni formali.

Come?

- Agli occhi dell'utente: l'utente scrive la dimostrazione in un linguaggio che il theorem prover capisce. Se compila, la dimostrazione è giusta!
- Internamente: il theorem prover analizza la dimostrazione scritta dall'utente seguendo questi passi:

Come facciamo a formalizzare della matematica?

Un *proof assistant* o *theorem prover interattivo* è un programma per scrivere dimostrazioni formali.

Come?

- Agli occhi dell'utente: l'utente scrive la dimostrazione in un linguaggio che il theorem prover capisce. Se compila, la dimostrazione è giusta!
- Internamente: il theorem prover analizza la dimostrazione scritta dall'utente seguendo questi passi:
  - parsing;

Come facciamo a formalizzare della matematica?

Un *proof assistant* o *theorem prover interattivo* è un programma per scrivere dimostrazioni formali.

Come?

- Agli occhi dell'utente: l'utente scrive la dimostrazione in un linguaggio che il theorem prover capisce. Se compila, la dimostrazione è giusta!
- Internamente: il theorem prover analizza la dimostrazione scritta dall'utente seguendo questi passi:
  - parsing;
  - elaborazione;

Come facciamo a formalizzare della matematica?

Un *proof assistant* o *theorem prover interattivo* è un programma per scrivere dimostrazioni formali.

Come?

- Agli occhi dell'utente: l'utente scrive la dimostrazione in un linguaggio che il theorem prover capisce. Se compila, la dimostrazione è giusta!
- Internamente: il theorem prover analizza la dimostrazione scritta dall'utente seguendo questi passi:
  - parsing;
  - elaborazione;
  - esecuzione delle tattiche;

Come facciamo a formalizzare della matematica?

Un *proof assistant* o *theorem prover interattivo* è un programma per scrivere dimostrazioni formali.

Come?

- Agli occhi dell'utente: l'utente scrive la dimostrazione in un linguaggio che il theorem prover capisce. Se compila, la dimostrazione è giusta!
- Internamente: il theorem prover analizza la dimostrazione scritta dall'utente seguendo questi passi:
  - parsing;
  - elaborazione;
  - esecuzione delle tattiche;
  - kernel checking.

Name	Latest version	Developer(s)	Implementation language	Features					
				Higher-order logic	Dependent types	Small kernel	Proof automation	Proof by reflection	Code generation
ACL2	8.3	Matt Kaufmann and J Strother Moore	Common Lisp	No	Untyped	No	Yes	Yes[2]	Already executable
Agda	2.6.3	Ulf Norell, Nils Anders Danielsson, and Andreas Abel (Chalmers and Gothenburg)	Haskell	Yes	Yes	Yes	No	Partial	Already executable
Albatross	0.4	Helmut Brandl	OCaml	Yes	No	Yes	Yes	Unknown	Not yet Implemented
Coq	8.19.0	INRIA	OCaml	Yes	Yes	Yes	Yes	Yes	Yes
F*	repository	Microsoft Research and INRIA	F*	Yes	Yes	No	Yes	Yes[3]	Yes
HOL Light	repository	John Harrison	OCaml	Yes	No	Yes	Yes	No	No
HOL4	Kananaskis-13 (or repo)	Michael Norrish, Konrad Slind, and others	Standard ML	Yes	No	Yes	Yes	No	Yes
Idris	2 0.6.0.	Edwin Brady	Idris	Yes	Yes	Yes	Unknown	Partial	Yes

Name	Latest version	Developer(s)	Implementation language	Features					
				Higher-order logic	Dependent types	Small kernel	Proof automation	Proof by reflection	Code generation
Isabelle	Isabelle2024 (May 2024)	Larry Paulson (Cambridge), Tobias Nipkow (München) and Makarius Wenzel	Standard ML, Scala	Yes	No	Yes	Yes	Yes	Yes
Lean	v4.7.0[4]	Leonardo de Moura (Microsoft Research)	C++, Lean	Yes	Yes	Yes	Yes	Yes	Yes
LEGO (not affiliated with Lego)	1.3.1	Randy Pollack (Edinburgh)	Standard ML	Yes	Yes	Yes	No	No	No
Metamath	v0.198[5]	Norman Megill	ANSI C						
Mizar	8.1.11	Białystok University	Free Pascal	Partial	Yes	No	No	No	No
Nqthm									
NuPRL	5	Cornell University	Common Lisp	Yes	Yes	Yes	Yes	Unknown	Yes
PVS	6.0	SRI International	Common Lisp	Yes	Yes	No	Yes	No	Unknown
Twelf	1.7.1	Frank Pfenning and Carsten Schürmann	Standard ML	Yes	Yes	Unknown	No	No	Unknown

Lean: un po' di storia

- Rilasciato nel 2013 da Leonardo de Moura (Microsoft Research, ora AWS);

Lean: un po' di storia

- Rilasciato nel 2013 da Leonardo de Moura (Microsoft Research, ora AWS);
- Lean 3 è la prima versione stabile;

Lean: un po' di storia

- Rilasciato nel 2013 da Leonardo de Moura (Microsoft Research, ora AWS);
- Lean 3 è la prima versione stabile;
- La prima versione di `mathlib` è scritta in Lean 3;

Lean: un po' di storia

- Rilasciato nel 2013 da Leonardo de Moura (Microsoft Research, ora AWS);
- Lean 3 è la prima versione stabile;
- La prima versione di `mathlib` è scritta in Lean 3;
- Nel 2021 arriva Lean 4;

Lean: un po' di storia

- Rilasciato nel 2013 da Leonardo de Moura (Microsoft Research, ora AWS);
- Lean 3 è la prima versione stabile;
- La prima versione di `mathlib` è scritta in Lean 3;
- Nel 2021 arriva Lean 4;
- Nel 2023 la traduzione di `mathlib` in Lean 4 è completata;

Lean: un po' di storia

- Rilasciato nel 2013 da Leonardo de Moura (Microsoft Research, ora AWS);
- Lean 3 è la prima versione stabile;
- La prima versione di `mathlib` è scritta in Lean 3;
- Nel 2021 arriva Lean 4;
- Nel 2023 la traduzione di `mathlib` in Lean 4 è completata;
- Al momento `mathlib` conta oltre 210 000 teoremi e 100 000 definizioni.

Teoria dei tipi dipendenti

La logica di Lean si basa sulla *teoria dei tipi*, originariamente introdotta da Bertrand Russell nei primi del novecento come fondamento della matematica, per evitare la creazione di paradossi come nella *teoria degli insiemi*.

Teoria dei tipi dipendenti

La logica di Lean si basa sulla *teoria dei tipi*, originariamente introdotta da Bertrand Russell nei primi del novecento come fondamento della matematica, per evitare la creazione di paradossi come nella *teoria degli insiemi*.

Gli elementi fondamentali di questa logica sono i *tipi*, collezioni di oggetti detti *termini*.

Teoria dei tipi dipendenti

La logica di Lean si basa sulla *teoria dei tipi*, originariamente introdotta da Bertrand Russell nei primi del novecento come fondamento della matematica, per evitare la creazione di paradossi come nella *teoria degli insiemi*.

Gli elementi fondamentali di questa logica sono i *tipi*, collezioni di oggetti detti *termini*.

In teoria dei tipi *dipendenti*, si possono definire tipi che dipendono da altri tipi!

La libreria matematica di Lean

`mathlib` è una libreria open source di matematica formalizzata in Lean. Contiene matematica e codice, nonché la definizione di tattiche utili per formalizzare le dimostrazioni.

Alcuni settori formalizzati: algebra commutativa, topologia, analisi, teoria dei numeri, schemi, etc.

Risultati classici in Lean

Freek Wiedijk ha una lista (100 theorems in Lean) aggiornata per tenere traccia dei progressi dei proof assistant nella formalizzazione di 100 problemi classici:

- l'irrazionalità della radice quadrata di 2,

Risultati classici in Lean

Freek Wiedijk ha una lista (100 theorems in Lean) aggiornata per tenere traccia dei progressi dei proof assistant nella formalizzazione di 100 problemi classici:

- l'irrazionalità della radice quadrata di 2,
- il teorema fondamentale dell'algebra,

Risultati classici in Lean

Freek Wiedijk ha una lista (100 theorems in Lean) aggiornata per tenere traccia dei progressi dei proof assistant nella formalizzazione di 100 problemi classici:

- l'irrazionalità della radice quadrata di 2,
- il teorema fondamentale dell'algebra,
- la numerabilità dei numeri razionali,

Risultati classici in Lean

Freek Wiedijk ha una lista (100 theorems in Lean) aggiornata per tenere traccia dei progressi dei proof assistant nella formalizzazione di 100 problemi classici:

- l'irrazionalità della radice quadrata di 2,
- il teorema fondamentale dell'algebra,
- la numerabilità dei numeri razionali,
- il teorema di Pitagora,

Risultati classici in Lean

Freek Wiedijk ha una lista (100 theorems in Lean) aggiornata per tenere traccia dei progressi dei proof assistant nella formalizzazione di 100 problemi classici:

- l'irrazionalità della radice quadrata di 2,
- il teorema fondamentale dell'algebra,
- la numerabilità dei numeri razionali,
- il teorema di Pitagora,
- il teorema dei numeri primi,

Risultati classici in Lean

Freek Wiedijk ha una lista (100 theorems in Lean) aggiornata per tenere traccia dei progressi dei proof assistant nella formalizzazione di 100 problemi classici:

- l'irrazionalità della radice quadrata di 2,
- il teorema fondamentale dell'algebra,
- la numerabilità dei numeri razionali,
- il teorema di Pitagora,
- il teorema dei numeri primi,
- etc.

Risultati classici in Lean

Freek Wiedijk ha una lista (100 theorems in Lean) aggiornata per tenere traccia dei progressi dei proof assistant nella formalizzazione di 100 problemi classici:

- l'irrazionalità della radice quadrata di 2,
- il teorema fondamentale dell'algebra,
- la numerabilità dei numeri razionali,
- il teorema di Pitagora,
- il teorema dei numeri primi,
- etc.

Risultati classici in Lean

Freek Wiedijk ha una lista (100 theorems in Lean) aggiornata per tenere traccia dei progressi dei proof assistant nella formalizzazione di 100 problemi classici:

- l'irrazionalità della radice quadrata di 2,
- il teorema fondamentale dell'algebra,
- la numerabilità dei numeri razionali,
- il teorema di Pitagora,
- il teorema dei numeri primi,
- etc.

Al momento, 83 di questi teoremi sono completamente formalizzati in Lean.

Pierre de Fermat



Fonte: Wikipedia

Pierre de Fermat, studioso francese del Seicento, è noto (tra le altre cose) per i suoi contributi alla teoria dei numeri. Era solito scrivere annotazioni nel margine dei suoi libri, come l'*Arithmetica* di Diofanto.

Pierre de Fermat



Fonte: Wikipedia

Pierre de Fermat, studioso francese del Seicento, è noto (tra le altre cose) per i suoi contributi alla teoria dei numeri. Era solito scrivere annotazioni nel margine dei suoi libri, come l'*Arithmetica* di Diofanto.

Del suo *ultimo teorema* scrisse:
Ho una meravigliosa dimostrazione di questo teorema, che non entra nel margine stretto della pagina.

L'ultimo teorema di Fermat

Sia $n \geq 3$. Esistono tre numeri interi non nulli x, y, z tali che

$$x^n + y^n = z^n?$$

L'ultimo teorema di Fermat

Sia $n \geq 3$. Esistono tre numeri interi non nulli x, y, z tali che

$$x^n + y^n = z^n?$$

- Per $n = 1$, è banale.

L'ultimo teorema di Fermat

Sia $n \geq 3$. Esistono tre numeri interi non nulli x, y, z tali che

$$x^n + y^n = z^n?$$

- Per $n = 1$, è banale.
- Per $n = 2$, ci sono infinite soluzioni: le terne pitagoriche.

L'ultimo teorema di Fermat

Sia $n \geq 3$. Esistono tre numeri interi non nulli x, y, z tali che

$$x^n + y^n = z^n?$$

- Per $n = 1$, è banale.
- Per $n = 2$, ci sono infinite soluzioni: le terne pitagoriche.
- Ma per $n \geq 3$, questo problema è rimasto aperto per 350 anni.

Un progetto molto ambizioso

Andrew Wiles pubblica la dimostrazione dell'ultimo teorema di Fermat nel 1994. Gli argomenti necessari utilizzano della matematica che non era ancora nota ai tempi di Fermat:

Un progetto molto ambizioso

Andrew Wiles pubblica la dimostrazione dell'ultimo teorema di Fermat nel 1994. Gli argomenti necessari utilizzano della matematica che non era ancora nota ai tempi di Fermat:

- curve ellittiche

Un progetto molto ambizioso

Andrew Wiles pubblica la dimostrazione dell'ultimo teorema di Fermat nel 1994. Gli argomenti necessari utilizzano della matematica che non era ancora nota ai tempi di Fermat:

- curve ellittiche
- forme modulari

Un progetto molto ambizioso

Andrew Wiles pubblica la dimostrazione dell'ultimo teorema di Fermat nel 1994. Gli argomenti necessari utilizzano della matematica che non era ancora nota ai tempi di Fermat:

- curve ellittiche
- forme modulari
- rappresentazioni di Galois

Un progetto molto ambizioso

Andrew Wiles pubblica la dimostrazione dell'ultimo teorema di Fermat nel 1994. Gli argomenti necessari utilizzano della matematica che non era ancora nota ai tempi di Fermat:

- curve ellittiche
- forme modulari
- rappresentazioni di Galois

Un progetto molto ambizioso

Andrew Wiles pubblica la dimostrazione dell'ultimo teorema di Fermat nel 1994. Gli argomenti necessari utilizzano della matematica che non era ancora nota ai tempi di Fermat:

- curve ellittiche
- forme modulari
- rappresentazioni di Galois

Nel 2024, il matematico inglese Kevin Buzzard ha vinto un grant di ricerca con lo scopo di formalizzare interamente la dimostrazione dell'ultimo teorema di Fermat. Il suo lavoro è open source e chiunque può collaborare.

Possiamo formalizzare la ricerca moderna in Lean?

(Una timeline liberamente adattata da questo seminario)

- Nei primi anni di vita dei proof assistant, questi venivano usati solo dagli informatici (per ricercare bug nei codici).

Possiamo formalizzare la ricerca moderna in Lean?

(Una timeline liberamente adattata da questo seminario)

- Nei primi anni di vita dei proof assistant, questi venivano usati solo dagli informatici (per ricercare bug nei codici).
- Nel 2004, Avigad et al. formalizzano il teorema dei numeri primi in Isabelle.

Possiamo formalizzare la ricerca moderna in Lean?

(Una timeline liberamente adattata da questo seminario)

- Nei primi anni di vita dei proof assistant, questi venivano usati solo dagli informatici (per ricercare bug nei codici).
- Nel 2004, Avigad et al. formalizzano il teorema dei numeri primi in Isabelle.
- Fino a una decina di anni fa, la matematica formalizzata era relativamente vecchia e riguardava per lo più concetti semplici (dimostrazioni di teoremi facili, eventualmente tediose).

Possiamo formalizzare la ricerca moderna in Lean?

(Una timeline liberamente adattata da questo seminario)

- Nei primi anni di vita dei proof assistant, questi venivano usati solo dagli informatici (per ricercare bug nei codici).
- Nel 2004, Avigad et al. formalizzano il teorema dei numeri primi in Isabelle.
- Fino a una decina di anni fa, la matematica formalizzata era relativamente vecchia e riguardava per lo più concetti semplici (dimostrazioni di teoremi facili, eventualmente tediose).
- Nel 2017, Buzzard suggerisce ai suoi studenti del primo anno di formalizzare la definzione di schema (ci riescono in due mesi).

Possiamo formalizzare la ricerca moderna in Lean?

(Una timeline liberamente adattata da questo seminario)

- Nei primi anni di vita dei proof assistant, questi venivano usati solo dagli informatici (per ricercare bug nei codici).
- Nel 2004, Avigad et al. formalizzano il teorema dei numeri primi in Isabelle.
- Fino a una decina di anni fa, la matematica formalizzata era relativamente vecchia e riguardava per lo più concetti semplici (dimostrazioni di teoremi facili, eventualmente tediose).
- Nel 2017, Buzzard suggerisce ai suoi studenti del primo anno di formalizzare la definzione di schema (ci riescono in due mesi).
- Nello stesso anno, Buzzard e Massot iniziano a lavorare alla formalizzazione degli spazi perfettoidi e al liquid tensor experiment di Scholze: finalmente la formalizzazione di un concetto molto complicato.

Colpo di scena!

- 2017: Ellenberg e Gijswijt dimostrano the Cap-Set Conjecture.

Colpo di scena!

- 2017: Ellenberg e Gijswijt dimostrano the Cap-Set Conjecture.
- 2019: Dahmen, Hoelzl e Lewis formalizzano la dimostrazione.

Colpo di scena!

- 2017: Ellenberg e Gijswijt dimostrano the Cap-Set Conjecture.
- 2019: Dahmen, Hoelzl e Lewis formalizzano la dimostrazione.
- 2021: Bloom dimostra una vecchia congettura di Erdos e Davenport. Appena sei mesi dopo, lui e Mehta formalizzano la dimostrazione.

Colpo di scena!

- 2017: Ellenberg e Gijswijt dimostrano the Cap-Set Conjecture.
- 2019: Dahmen, Hoelzl e Lewis formalizzano la dimostrazione.
- 2021: Bloom dimostra una vecchia congettura di Erdos e Davenport. Appena sei mesi dopo, lui e Mehta formalizzano la dimostrazione.
- 2023: Campos, Griffiths, Morris e Sahasrabudhe annunciano nuovi risultati sui numeri di Ramsey. Mehta formalizza la dimostrazione in cinque mesi.

Colpo di scena!

- 2017: Ellenberg e Gijswijt dimostrano the Cap-Set Conjecture.
- 2019: Dahmen, Hoelzl e Lewis formalizzano la dimostrazione.
- 2021: Bloom dimostra una vecchia congettura di Erdos e Davenport. Appena sei mesi dopo, lui e Mehta formalizzano la dimostrazione.
- 2023: Campos, Griffiths, Morris e Sahasrabudhe annunciano nuovi risultati sui numeri di Ramsey. Mehta formalizza la dimostrazione in cinque mesi.
- 2023: Terry Tao impara a usare Lean.

Colpo di scena!

- 2017: Ellenberg e Gijswijt dimostrano the Cap-Set Conjecture.
- 2019: Dahmen, Hoelzl e Lewis formalizzano la dimostrazione.
- 2021: Bloom dimostra una vecchia congettura di Erdos e Davenport. Appena sei mesi dopo, lui e Mehta formalizzano la dimostrazione.
- 2023: Campos, Griffiths, Morris e Sahasrabudhe annunciano nuovi risultati sui numeri di Ramsey. Mehta formalizza la dimostrazione in cinque mesi.
- 2023: Terry Tao impara a usare Lean.
- 2023: Gowers, Green, Manners e Tao annunciano una dimostrazione della congettura di Freiman–Ruzsa. 25 persone collaborano alla formalizzazione del risultato, che sarà pronta in tre settimane (prima ancora della submission del paper!).

Colpo di scena!

- 2017: Ellenberg e Gijswijt dimostrano the Cap-Set Conjecture.
- 2019: Dahmen, Hoelzl e Lewis formalizzano la dimostrazione.
- 2021: Bloom dimostra una vecchia congettura di Erdos e Davenport. Appena sei mesi dopo, lui e Mehta formalizzano la dimostrazione.
- 2023: Campos, Griffiths, Morris e Sahasrabudhe annunciano nuovi risultati sui numeri di Ramsey. Mehta formalizza la dimostrazione in cinque mesi.
- 2023: Terry Tao impara a usare Lean.
- 2023: Gowers, Green, Manners e Tao annunciano una dimostrazione della congettura di Freiman–Ruzsa. 25 persone collaborano alla formalizzazione del risultato, che sarà pronta in tre settimane (prima ancora della submission del paper!).
- Idealmente, un ricercatore vorrebbe avere per ogni suo articolo una formalizzazione annessa che ne certifichi la correttezza.

E l'intelligenza artificiale ci può aiutare?

Da un mio vecchio (ma non tanto) talk:

E l'intelligenza artificiale ci può aiutare?

Da un mio vecchio (ma non tanto) talk:

Is AI useful in formalising mathematics?

Short answer: probably.

- Present: AI tools such as Copilot help autocomplete programming code (including Lean).
- Short term future: Develop a tool to translate LaTeX to Lean.
- Long term future: AI may become better than humans at *finding* a proof.

Figura: è già invecchiata questa slide!

State of the art

- 2022: OpenAI e Meta AI creano modelli AI per generare dimostrazioni e soluzioni di problemi delle olimpiadi della matematica in Lean.

State of the art

- 2022: OpenAI e Meta AI creano modelli AI per generare dimostrazioni e soluzioni di problemi delle olimpiadi della matematica in Lean.
- 2023: Nasce Harmonic, una startup che si pone l'obiettivo di ridurre le allucinazioni dell'AI generando (e controllando) codice Lean.

State of the art

- 2022: OpenAI e Meta AI creano modelli AI per generare dimostrazioni e soluzioni di problemi delle olimpiadi della matematica in Lean.
- 2023: Nasce Harmonic, una startup che si pone l'obiettivo di ridurre le allucinazioni dell'AI generando (e controllando) codice Lean.
- 2024: Google DeepMind crea AlphaProof, che ottiene una medaglia d'argento alle IMO (International Mathematical Olympiad).

State of the art

- 2022: OpenAI e Meta AI creano modelli AI per generare dimostrazioni e soluzioni di problemi delle olimpiadi della matematica in Lean.
- 2023: Nasce Harmonic, una startup che si pone l'obiettivo di ridurre le allucinazioni dell'AI generando (e controllando) codice Lean.
- 2024: Google DeepMind crea AlphaProof, che ottiene una medaglia d'argento alle IMO (International Mathematical Olympiad).
- 2025: DeepSeek introduce DeepSeek-Prover-V2, un modello AI per scrivere dimostrazioni in Lean 4.

State of the art

- 2022: OpenAI e Meta AI creano modelli AI per generare dimostrazioni e soluzioni di problemi delle olimpiadi della matematica in Lean.
- 2023: Nasce Harmonic, una startup che si pone l'obiettivo di ridurre le allucinazioni dell'AI generando (e controllando) codice Lean.
- 2024: Google DeepMind crea AlphaProof, che ottiene una medaglia d'argento alle IMO (International Mathematical Olympiad).
- 2025: DeepSeek introduce DeepSeek-Prover-V2, un modello AI per scrivere dimostrazioni in Lean 4.
- 2025: Rappresentanti di molte di queste aziende partecipano ad ItaLean (Bologna)

Grazie!

